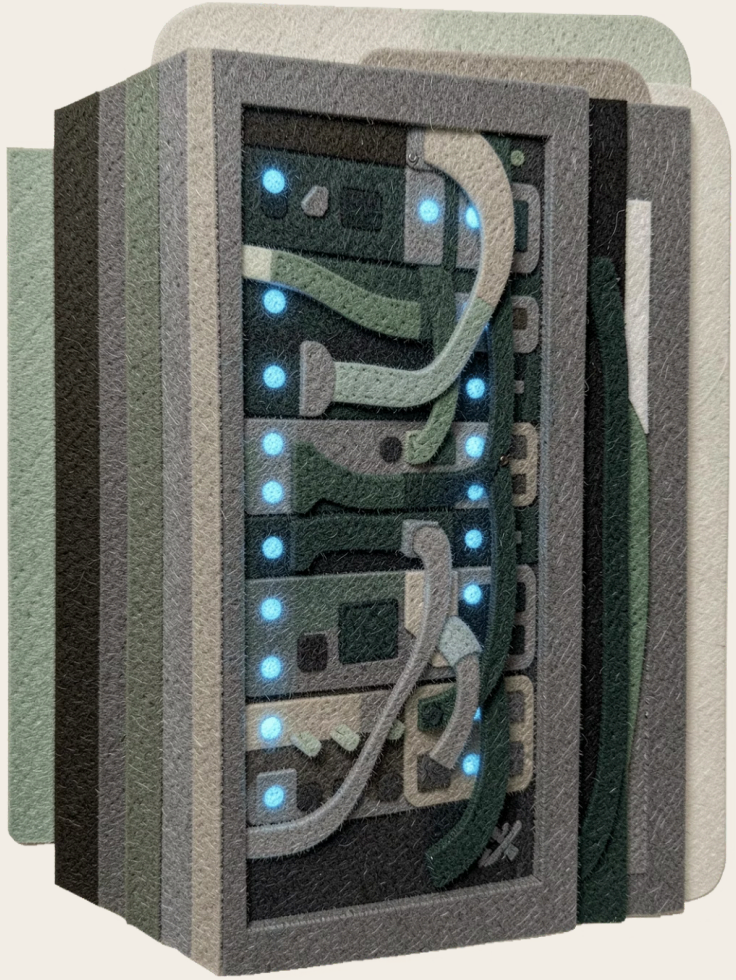
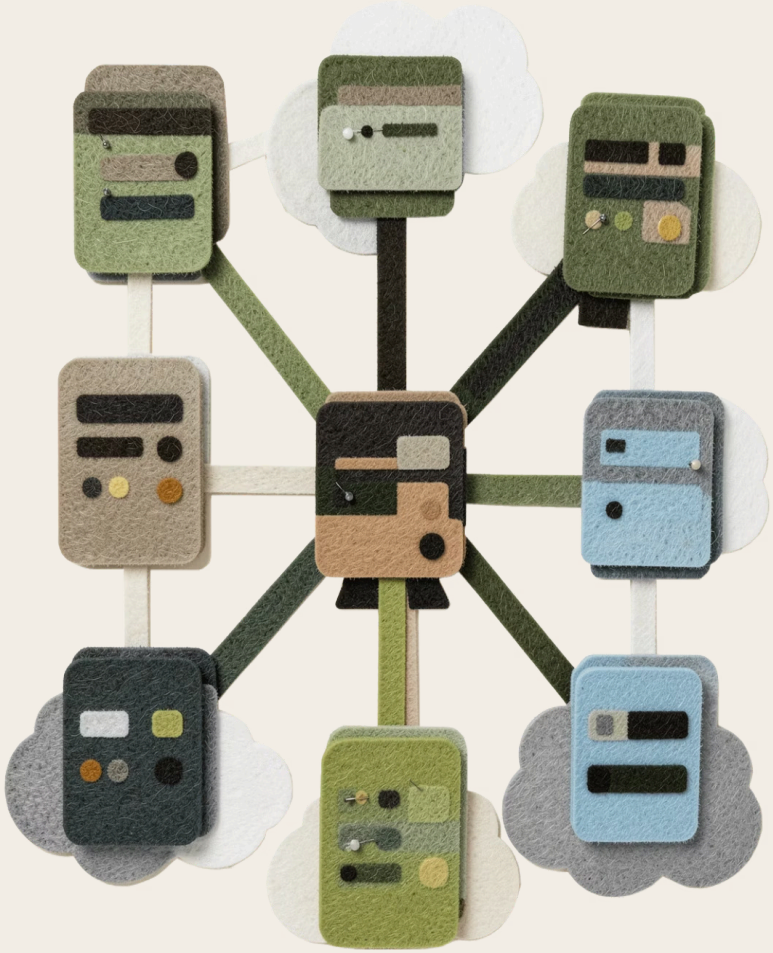
Operational Detection of Guest-to-Host Compromise via QEMU Process Telemetry

Josh Buckwald

CS-GY 6813 INFORMATION SECURITY & PRIVACY

SPRING 2026





Virtualization: The Foundation of Modern Infrastructure

Cloud providers like AWS and Google operate massive data centers, pooling compute, memory, and storage resources and renting them on demand as **Virtual Machines**. This model transformed how quickly companies deploy and scale — launching infrastructure in minutes rather than months, at a fraction of the cost of maintaining physical hardware.

Resource Pooling

Physical servers supply shared compute, memory, and storage to many VMs simultaneously, maximizing utilization.

On-Demand Scaling

Workloads scale up or down in minutes, supporting global operations without dedicated hardware.

Isolation Promise

Each VM behaves as its own isolated system — even while sharing physical hardware with others.

The Threat Model: What Happens When Isolation Fails

The security of multi-tenant cloud infrastructure **depends entirely on VM isolation.** If an attacker escapes the guest VM into the hypervisor layer — the **host** — the consequences are catastrophic and far-reaching.

Single Hypervisor Impact

- Denial of service across every tenant on that hypervisor
- Lateral access to all co-resident VMs and their data
- Complete compromise of the shared hardware layer

Datacenter-Wide Escalation

- Threat actors pivot between hypervisors across a datacenter
- Impact multiplies across dozens or hundreds of organizations
- Recovery becomes exponentially more complex and costly

⚠ A single VM escape can cascade into a multi-tenant breach affecting entire business ecosystems.

VM Escape Is Not Theoretical

VM Escape — also known as Guest-to-Host Compromise — has been documented in both research settings and in the wild. Despite its severity, **operational detection remains an unsolved problem**.

The Core Problem

Defenders must assume that telemetry originating from the guest OS is **untrustworthy**. An attacker with sufficient control can manipulate logs, suppress signals, and obscure activity entirely.

The Detection Dilemma

How do you defend a system when you cannot trust the data it produces? Traditional endpoint detection, log analysis, and SIEM correlation all rely on guest-provided signals — exactly what an attacker would compromise first.

Related Research: Virtual Machine Introspection

In 2003, Garfinkel and Rosenblum introduced **Virtual Machine Introspection (VMI)** — a concept that monitors low-level hardware signatures from the hypervisor to reconstruct high-level guest activity, without relying on the guest itself.

1 Hardware-Level Signatures

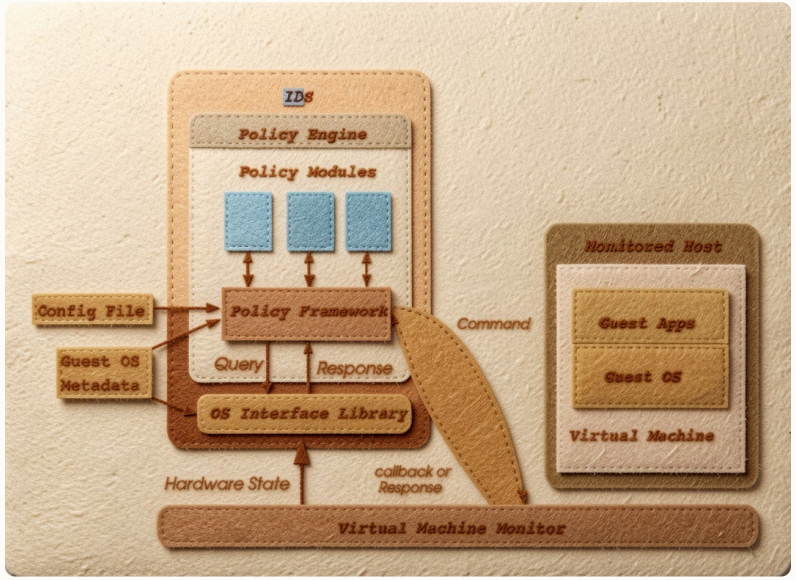
Every OS-level action leaves a trace at the hardware layer — memory changes, disk writes, register states — invisible to guest tampering.

2 The Semantic Gap

Translating raw hardware signals into meaningful activity requires significant compute and analytical overhead, limiting real-time utility.

3 Forensic, Not Operational

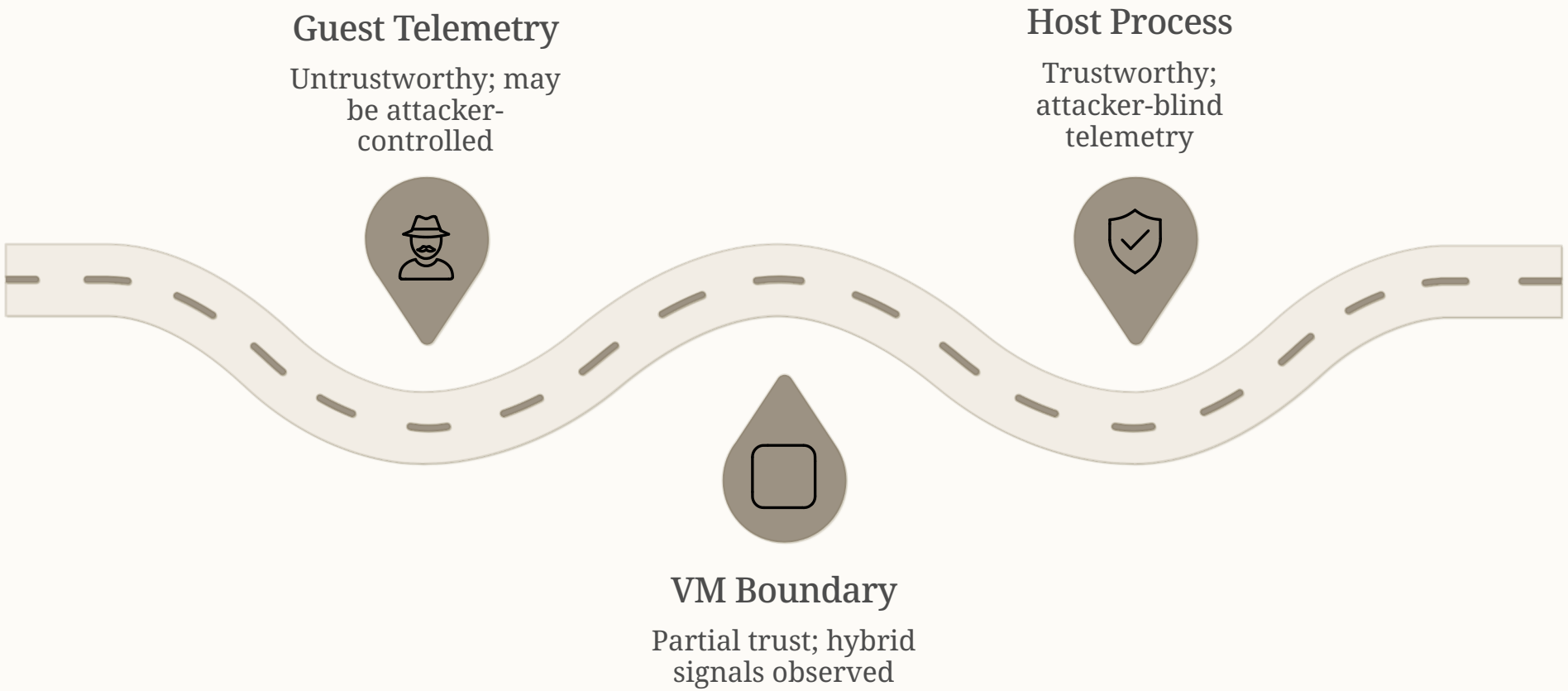
VMI excels at post-incident analysis but cannot reliably detect and stop VM escape before damage occurs.



VMI is like the raw readout of a polygraph test — it catches lies, but too late to prevent harm.

Hypothesis: Telemetry at the VM Boundary Can Be Trusted

The foundational assumption — that *all* guest telemetry is untrustworthy — may be too broad. At the **boundary between guest and host**, we can observe host-side effects directly caused by guest activity.



By focusing on a process that straddles both worlds — executing guest instructions while running as a host process — we can observe signals that are both **operationally accessible** and **resistant to guest-side manipulation**.

QEMU: The Perfect Candidate for Operational Detection

The **Quick Emulator (QEMU)** process runs on the host and is responsible for executing every instruction the VM issues. It sits precisely at the guest/host boundary — making it an ideal source of trustworthy telemetry.

Boundary Position

QEMU executes guest instructions as host processes — any compromise must pass through it, leaving observable traces.

Predictable Behavior

Under normal conditions, QEMU's behavior is highly consistent. Unexpected child processes or sensitive resource access are strong anomaly signals.

Lightweight Detection

Monitoring QEMU via host-level audit tools requires minimal overhead compared to full VMI — enabling real-time, operational alerting.

Experiment Design

To test the hypothesis, a controlled experiment was designed to simulate **post-exploitation behavior** at the QEMU boundary — without exploiting a real vulnerability. The goal was to observe whether host-level monitoring could detect anomalous QEMU behavior in real time.

Environment

- VM running through QEMU on a Linux host
- Host-level monitoring via **auditd**
- Process execution and sensitive resource access logged

Primary Metrics

- **Mean Time to Detection (MTTD)** — how quickly signals appear after compromise
- **LM Cyber Kill Chain Phase** — at what stage does detection occur relative to attacker progress

Approach

Simulate command execution within QEMU to mimic post-escape behavior, then measure host-level signal clarity and detection speed.

Experiment Execution: Simulating Post-Exploitation

With the environment configured, the experiment triggered **command execution within the QEMU process itself** — simulating what an attacker would do after successfully crossing the guest/host boundary. The focus was on behaviors most likely to generate observable host-level signals.

1

Trigger

Command execution injected into QEMU process context, mimicking successful VM escape

2

Monitor

auditd captures process spawns, privilege levels, terminal associations, and resource access in real time

3

Detect

Anomalous signals identified: unexpected shell execution, root privileges, no associated terminal

4

Alert

Signals mapped to actionable detection rules suitable for operational SOC deployment

Key Audit Log Entries Observed During Experiment

These three log entries together show unauthorized command execution occurring on the host through the QEMU process.

1 PROCESS EXECUTION (THE TRIGGER)

A shell was launched to execute a command. This is the initial signal of compromiee.

```
type=EXECVE msg=audit(1775966363.687:642):
argc=4 a0="sh" a1="-c"
a2="id > /tmp/owned" a3=(null)
exe="/usr/bin/dash" subj=unconfined
key=(null)
```

What this means:

QEMU executed a shell command: sh -c

This is not normal behavior

2 KERNEL CONFIRMATION (IT ACTUALLY HAPPENED)

Confirms at the kernel level that the command executed successfully as root, without a terminal — highly abnormal

```
type=SYSCALL msg=audit(1775966363.687:642):
arch=c000003e syscall=221 success=yes
exit=0 a0=7ffeb1a1f020 a1=7ffeb1a1fe10
a2=7ffeb1a1fd08 a3=0 items=0 ppid=1868
pid=2116 auid=4294967295 uid=0 gid=0
tty=(none) ses=4294967295 comm="sh" exe="/usr/bin/dash"
```

What this means:

- syscall=221 → execve (a new process was executed)
- success=yes → it succeeded
- uid=0 → ran with root privileges
- key=(none) → no terminal (non-interactive)

3 PROCESS IDENTITY (WHAT ACTUALLY RAN)

Shows the actual binary that was executed, confirming a real shell ran on the host.

```
type=PATH msg=audit(1775966363.687:642):
item=0 name="/usr/bin/dash" inode=131074
dev=08:01 mode=0100755 ouid=0 ogid=0
rdev=00:00 obj=system_u:object_r:bin_t:s0
nametype=NORMAL cap_fp=1 cap_fi=1 cap_fe=1
cap_fver=0
```

What this means:

The command was executed using /usr/bin/dash (a shell), owned by root.

Takeaway: *QEMU executed a shell command as root, without a terminal, which is a strong indicator of guest-to-host compromise.*

Results: Clear, Immediate Signals at the Host Level

The simulated attack produced **unambiguous host-level signals**. Audit logs revealed a shell process spawning with **root privileges and no associated terminal** — behavior that is categorically abnormal for QEMU under normal operation.

QEMU Telemetry Detection

MTTD: Near-instantaneous — detection at the **Exploitation phase** of the Cyber Kill Chain

Direct, interpretable signals requiring no semantic translation layer

VMI Comparison

MTTD: Minutes to hours — typically detection at **Actions on Objectives phase**

Requires significant analytical overhead to interpret low-level hardware signatures

~0s	1	3
Mean Time to Detection	**Kill Chain Phase Earlier**	**Key Anomaly Signals**
QEMU telemetry provides near-immediate signal upon compromise activity	Detection at Exploitation phase — before attacker achieves objectives	Shell spawn, root privilege, no terminal — all observable via auditd

The hypothesis is supported: monitoring QEMU process behavior provides a lightweight, operationally viable method for detecting guest-to-host compromise at the moment it occurs.

Further Research

Expand Detection Scope

This method works well against narrow VM escape types. Further research needed to identify other boundary processes like QEMU that can be reliably monitored.

Real-World Exploit Validation

Test against actual modern exploits designed to evade detection. Validate that detection signals hold up against real-world attack scenarios.

Production Integration

Integrate into real-time monitoring systems using eBPF-based tooling or SIEM platforms. Evaluate performance at scale in production environments.

Complementary Approach

Combine QEMU telemetry with Virtual Machine Introspection. Use early detection from QEMU with deeper analysis from VMI, not as replacement but as complementary defense.

References

1. P. Mishra, E. S. Pilli, V. Varadharajan, and U. Tupakula, "Intrusion detection techniques in cloud environment: A survey," Journal of Network and Computer Applications, vol. 77, pp. 18–47, 2017.
2. T. Garfinkel and M. Rosenblum, "A virtual machine introspection based architecture for intrusion detection," in Proceedings of the Network and Distributed Systems Security Symposium (NDSS), 2003.
3. K. Zhang, J. Ma, and C. Li, "Using KVM events to detect VM memory-sharing lateral movement attacks in a virtualized environment," in 2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), 2024, doi: 10.1109/ISPA63168.2024.00150.